

Отчёт по домашней работе 4

Технический дизайн микросервисной архитектуры

Милютин Никита, БР1.2

1. Тема

Проектирование микросервисной архитектуры для приложения бронирования столиков в ресторанах.

2. Цель работы

Разделить монолитное backend-приложение на независимые микросервисы с подходом database-per-service, описать связи между сервисами, варианты межсервисного взаимодействия, разделение баз данных и OpenAPI-контракты.

3. Исходная система

Исходное приложение реализует REST API для бронирования столиков в ресторанах. В предметной области есть пользователи, рестораны, кухни, столики, бронирования, отзывы, фотографии и файлы меню.

В монолите эти части находятся в одном приложении. При переходе к микросервисной архитектуре каждая бизнес-область выделяется в отдельный сервис и получает собственную базу данных.

4. Состав микросервисов

Сервис	Ответственность	База данных
API Gateway	Единая внешняя точка входа, маршрутизация, проверка JWT	нет
Auth Service	Регистрация, вход, профиль пользователя	auth_db
Restaurant Service	Рестораны, кухни, связи ресторанов и кухонь	restaurant_db
Table Service	Столики ресторанов и их доступность	table_db
Reservation Service	Создание, просмотр и отмена бронирований	reservation_db

Review Service	Отзывы и рейтинги ресторанов	review_db
Media Service	Фото ресторанов и файлы меню	media_db
Notification Service	Уведомления о создании и отмене брони	notification_db

5. Разделение баз данных

Архитектура использует принцип database-per-service. Каждый сервис владеет только своей схемой данных, а другие сервисы не имеют прямого доступа к его таблицам.

- auth_db: таблица users.
- restaurant_db: таблицы restaurants, cuisines, restaurant_cuisines.
- table_db: таблица tables.
- reservation_db: таблица reservations.
- review_db: таблица reviews.
- media_db: таблицы restaurant_photos, restaurant_menus.
- notification_db: таблицы notifications, delivery_attempts.

Связи между сущностями разных сервисов хранятся как идентификаторы. Например, Reservation Service хранит user_id, restaurant_id и table_id, но не использует foreign key на чужие базы данных.

6. Межсервисное взаимодействие

Для синхронных операций используется REST:

- API Gateway вызывает Auth Service для регистрации, входа и профиля.
- API Gateway вызывает Restaurant Service для списка и карточки ресторанов.
- API Gateway вызывает Table Service для получения столиков.
- API Gateway вызывает Reservation Service для операций с бронями.
- Reservation Service вызывает Restaurant Service для проверки ресторана.
- Reservation Service вызывает Table Service для проверки столика и вместимости.

Для асинхронных операций используются события:

- reservation.created публикуется после создания бронирования;
- reservation.cancelled публикуется после отмены бронирования;
- Notification Service подписывается на эти события и создаёт уведомления.

7. Основной сценарий бронирования

1. Клиент отправляет запрос POST /api/reservations в API Gateway.
2. Gateway проверяет JWT и передаёт запрос в Reservation Service.

3. Reservation Service проверяет существование ресторана через Restaurant Service.
4. Reservation Service проверяет столик через Table Service.
5. Reservation Service проверяет отсутствие активной брони на выбранное время.
6. Reservation Service создаёт запись в reservation_db.
7. Reservation Service публикует событие reservation.created.
8. Notification Service создаёт уведомление пользователю.

8. OpenAPI-контракты

В файле internal-openapi.yaml описаны внутренние REST-контракты:

- GET /auth/internal/users/{userId} — получение пользователя по идентификатору;
- GET /restaurants/internal/restaurants/{restaurantId} — проверка ресторана;
- GET /tables/internal/restaurants/{restaurantId}/tables/{tableId} — проверка столика;
- GET /reservations/internal/tables/{tableId}/availability — проверка доступности столика;
- POST /reservations — создание бронирования через Gateway.

Для ответов описаны успешные схемы и ошибки 400, 401, 404, 409, 503.

9. Обработка ошибок

Код	Причина
400	Некорректные данные запроса
401	Отсутствует или неверен JWT
403	Попытка изменить чужой ресурс
404	Ресторан, столик, пользователь или бронь не найдены
409	Столик уже забронирован на выбранное время
503	Зависимый сервис временно недоступен

Для межсервисных запросов необходимо использовать таймауты, correlation id в заголовке X-Request-Id и ретрай только для идемпотентных операций.

10. План миграции

1. Выделить API Gateway, сохранив внешние маршруты.
2. Вынести Auth Service и таблицу users.
3. Вынести Restaurant Service вместе с кухнями.
4. Вынести Table Service и Media Service.

5. Вынести Reservation Service и заменить прямые обращения к данным REST-вызовами.
6. Вынести Review Service.
7. Добавить брокер событий для уведомлений.
8. Удалить прямые обращения к чужим базам данных.

11. Результат

В результате работы подготовлен технический дизайн микросервисной архитектуры для приложения бронирования ресторанов. Описано разбиение монолита на сервисы, разделение баз данных, синхронное и асинхронное взаимодействие, возможные ошибки и OpenAPI-спецификация внутренних контрактов.